

Abc453 D - Go Straight

题意简述

给定一张 $N \times M$ 的网格地图，由以下字符构成：

- **#**: 障碍，不可通行。
- **.**: 空地，可任意通行。
- **o**: 特殊点，进入后只能沿着**进入的方向**继续直线移动。
- **x**: 特殊点，进入后只能沿着**非进入的方向**（即其他三个方向）移动。
- **S**: 起点，可任意通行。
- **G**: 终点，可任意通行。

判断能否从起点到达终点。如果可以，需要给出一个长度小于 5×10^6 的移动序列（由 **U**, **D**, **L**, **R** 组成）。

解题思路

这是一个规则稍复杂的 BFS 最短路问题。

关键点在于状态设计：除了位置 (r, c) 外，还需要记录到达此位置时的“朝向” d （0,1,2,3 分别代表上、下、左、右）。

在 BFS 过程中，根据当前格子类型和朝向 d ，决定下一步可以走的方向：

1. 如果当前格子是 **S**, **.** 或 **G**，或者（起点），则可以向任意四个方向移动。
2. 如果当前格子是 **o**，则只能沿原方向 d 移动。
3. 如果当前格子是 **x**，则可以向除了 d 以外的其他三个方向移动。

在 BFS 过程中记录前驱状态，用于最后回溯构造路径。

由于状态数为 $O(NM)$ ，每条边（状态转移）最多产生 4 个新状态，BFS 复杂度为 $O(NM)$ 。

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1010;
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};
char dir[4] = {'U', 'D', 'L', 'R'};
string s[N];
int vis[N][N][4], pre[N][N][4];
//pre[x][y][d] -> 以 d 方向进入 (x,y) 时候，上一个状态是什么
```

```

int main() {
    int n, m;
    cin >> n >> m;
    int sx, sy;
    for(int i = 1; i <= n; i++){
        cin >> s[i];
        s[i] = " " + s[i];
        for(int j = 1; j <= m; j++){
            if(s[i][j] == 'S'){
                sx = i, sy = j;
            }
        }
    }

    queue<array<int, 3>>q; // {行, 列, 方向}
    for(int d = 0; d < 4; d++){
        int nx = sx + dx[d], ny = sy + dy[d];
        if(nx < 1 || ny < 1 || nx > n || ny > m || s[nx][ny] == '#') continue;
        vis[nx][ny][d] = 1;
        pre[nx][ny][d] = -1;
        q.push({nx, ny, d});
    }

    while(!q.empty()){
        int x, y, d;
        x = q.front()[0];
        y = q.front()[1];
        d = q.front()[2];
        q.pop();
        if(s[x][y] == 'G'){
            string ans = "";
            int cx = x, cy = y, cd = d;
            while (cd != -1) {
                ans += dir[cd];
                int pd = pre[cx][cy][cd];
                cx -= dx[cd];
                cy -= dy[cd];
                cd = pd;
            }
            cout << "Yes" << endl;
            reverse(ans.begin(), ans.end());
            cout << ans;
            return 0;
        }

        for (int nd = 0; nd < 4; nd++) {
            if (s[x][y] == 'o' && nd != d) continue; // 必须保持同向

```

```

        if (s[x][y] == 'x' && nd == d) continue; // 必须改变方向
        int nx = x + dx[nd];
        int ny = y + dy[nd];
        if(nx < 1 || ny < 1 || nx > n || ny > m || s[nx][ny] == '#')
continue;

        if (vis[nx][ny][nd]) continue;
        vis[nx][ny][nd] = true;
        pre[nx][ny][nd] = d; // 记录是从哪个方向过渡来的
        q.push({nx, ny, nd});
    }
}
cout << "No";
return 0;
}

```

Abc453 E - Team Division

题意简述

有 N 个人，要分成两组 **A** 和 **B**。规则如下：

- 每队至少一人。
- 每个人必须属于恰好一队。
- 第 i 个人所在队伍的人数必须在 $[L_i, R_i]$ 区间内。

求合法的分组方案数，对 998244353 取模。

解题思路

考虑枚举 **A** 队的人数 a ，则 **B** 队人数为 $n-a$ 。

对于每个选手的合法去向可能是：

1. 只能去 **A** 队（仅满足 $a \in [l, r]$ 条件）。
2. 只能去 **B** 队（仅满足 $n - a \in [l, r]$ 条件）。
3. 两者均可（同时满足两个条件）。
4. 均不可（则该 a 不合法）。

对于所有的 a ，利用差分数组统计：

可以去 **A** 的人数： $cnt[a]$

可以去 B 的人数: $cnt[n - a]$

两者均可的人数: $cnt[a] + cnt[n - a] - n$

只能去 A 的人数: $n - cnt[n - a]$

此时方案数: $C(cnt[a] + cnt[n - a] - n, a - n + cnt[a])$

时间复杂度为 $O(N)$ 。

还需要保证枚举的 a 合法，其实对于每个人都有：

$$a \in [l, r] \text{ 或 } n - a \in [l, r]$$

可以写作两个区间的并，注意区间的左右顺序然后求交集即可。

```
#include<bits/stdc++.h>
using namespace std;

#define int long long
const int mod = 998244353, N = 2e5 + 5;
int fact[N], invf[N];
int fastpow(int a, int b)
{
    int res = 1;
    while (b) {
        if (b & 1) {
            res = (res * a) % mod;
        }
        b >>= 1;
        a = (a * a) % mod;
    }
    return res;
}
void init(int n)
{
    fact[0] = 1;
    for (int i = 1; i <= n + 1; i++) {
        fact[i] = (fact[i - 1] * i) % mod;
    }
    invf[n + 1] = fastpow(fact[n + 1], mod - 2);
    for (int i = n; i >= 0; i--) {
        invf[i] = (invf[i + 1] * (i + 1)) % mod;
    }
    return;
}
```

```

}
int C(int n, int m)
{
    return ((fact[n] * invf[m]) % mod * invf[n - m]) % mod;
}
int cnt[N]; //处理差分
signed main()
{
    int n;
    cin >> n;
    init(n);
    int L1=0, R1=n, L2=0, R2=n; //最终区间为 (L1,R1) 或 (L2,R2)
    for(int i=1; i<=n; i++)
    {
        int l, r;
        cin >> l >> r;
        cnt[l]++, cnt[r+1]--;
        //合法区间为 (l,r) 或 (n-r,n-l)
        int a=n-r, b=n-l;
        if(l>a)
        {
            l=a, r=b;
        }
        L1=max(L1, l), R1=min(R1, r);
        L2=max(L2, n-r), R2=min(R2, n-l);
    }
    for(int i=1; i<=n; i++) cnt[i]+=cnt[i-1]; //前缀和
    int ans=0;
    for(int a=1; a<n; a++)
    {
        if((L1<=a && a<=R1) || (L2<=a && a<=R2))
        {
            ans+=C(cnt[a]+cnt[n-a]-n, cnt[a]-a);
            ans%=mod;
        }
    }
    cout << ans << endl;
    return 0;
}

```

Abc454 E – LRUD Moving

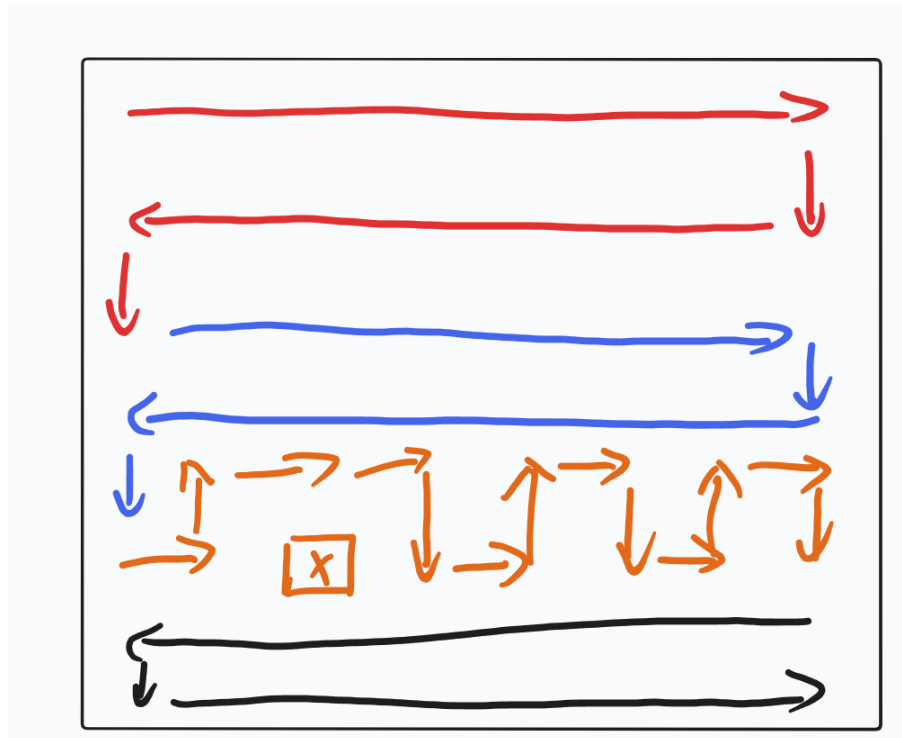
题意简述

$N \times N$ 网格图上，从左上角移动到右下角，必须经过除 (A,B) 以外的所有格子，输出一种移动方案。

解题思路

所有格子黑白交替染色，那么起点与终点颜色必然相同。若 N 为奇数，而移动步数 $N^2 - 2$ 为奇数，此时不存在合法方案。

若 N 为偶数，令起点终点均为黑色，所有格子中黑白数量相同，而移动过程中白色格子数量应该比黑色少 1，于是 (A,B) 必然是白色，即 $A+B$ 为奇数。由于 N 为偶数，每两行一起处理，对于包含障碍物的两行单独处理。



```
#include <bits/stdc++.h>
using namespace std;
void prn(char ch,int c) {
    for (int i=1;i<=c;i++) cout<<ch;
}
int main(){
    int T;
    cin>>T;
    while (T--) {
        int n,a,b;
        cin>>n>>a>>b;
```

```

    if (n%2==1 || (a+b)%2==0 ) {
        cout<<"No\n";continue;
    }
    cout<<"Yes\n";
    int m=0;
    while (m+2<a) {
        prn('R',n-1);cout<<'D';prn('L',n-1);cout<<'D';
        m+=2;
    }
    int k=0;
    while (k+2<b) {
        cout<<"DRUR";k+=2;
    }
    if (k+1==b) cout<<"RD";else cout<<"DR";k+=2;
    while (k<n) {
        cout<<"RURD";k+=2;
    }
    m+=2;
    while (m<n) {
        cout<<"D";prn('L',n-1);cout<<"D";prn('R',n-1);m+=2;
    }
    cout<<"\n";
}
return 0;
}

```

Abc454 F – Make it Palindrome 2

题意简述

长度为 N 的整数序列，每个元素都在 $0 \sim M-1$ 之间，执行任意次以下操作：
选择区间 $[l,r]$ ，将其中的每个数都加 $1 \pmod M$

求将 A 变为回文序列的最少操作次数。

解题思路

显然只需要修改前一半的元素，使它与后面对应位置的元素相等，计算对应位置的差值(数组 B)，问题变为通过加或减一的区间操作将所有元素变为 0。

考虑数组 B 的差分 D ，区间操作变为两个点位上一加一减，如果不考虑取模，最终差分数组全部为 0，而所有操作不改变其总和，因此答案为所有正数之和。

在模 M 意义下，数组 B 可以随意加减 M ，相当于差分数组 D 中一个值加 M ，另外一个值减 M ，对于原先两个数 D_1 (正)， D_2 (负) 可变为 D_1-M ， D_2+M 原先代价为 D_1 ，变化后代价为 D_2+M

贪心策略：

分离 D 数组：正数放入 P，负数放入 N。P 降序排列，N 升序排列。

配对 $P[i]$ 与 $N[i]$ ，若 $M+N[i]<P[i]$ （收益为负），则执行调整，累加收益。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int T;
    cin >> T;
    while (T--) {
        int n, m;
        cin >> n >> m;
        int k = n / 2;
        vector<long long> b(k + 2, 0); // 存储对称差 b_i

        for (int i = 1; i <= n; ++i) {
            int x;
            cin >> x;
            if (i <= k) {
                b[i] = x;
            } else if (i > n - k) { // 处理右半部分
                int j = n - i + 1;
                b[j] = (b[j] - x + m) % m;
            }
        }

        // 计算差分数组 d
        vector<long long> pos, neg;
        long long sum = 0;
        for (int i = k + 1; i >= 1; --i) { // 倒序避免覆盖
            b[i] -= b[i - 1];
            if (b[i] > 0) {
                pos.push_back(b[i]);
                sum += b[i];
            } else if (b[i] < 0) {
                neg.push_back(b[i]);
            }
        }

        // 贪心配对
        sort(pos.rbegin(), pos.rend());
        sort(neg.begin(), neg.end());
        int len = min(pos.size(), neg.size());
```



```

        for (int i = 0; i < len; ++i) {
            if (m + neg[i] < pos[i]) {
                sum += m + neg[i] - pos[i];
            } else {
                break; // 后续无收益，提前退出
            }
        }

        cout << sum << '\n';
    }
    return 0;
}

```

P16327 线段

题意简述

给定 N 个可重复线段， q 次操作，每次给出区间 $[l,r]$ ，将原先所有的线段分割，计算线段总数。

解题思路

对于一个查询，相当于在 l 和 $r+1$ 各切一刀，每切一刀，原先左端点小于 l 且右端点大于 l 的线段就会产生贡献，注意一个点只能被切一次，因此给每个点位打上标记。

```

#include<bits/stdc++.h>
using namespace std;
constexpr int N=5e5+5;

int n,q,sl[N],sr[N],ans;
vector<int> qx;
int get(int x){
    int cl=lower_bound(sl+1,sl+1+n,x)-sl-1;
    int cr=lower_bound(sr+1,sr+1+n,x)-sr-1;
    return cl-cr;
}
int main(){
    cin>>n>>q;
    for(int i=1;i<=n;i++){
        cin>>sl[i]>>sr[i];
    }
    sort(sl+1,sl+1+n);

```

```
sort(sr+1,sr+1+n);
map<int,bool> flag;//标记
ans=n;
while(q--){
    int l,r;
    cin>>l>>r;
    if(!flag[l])
    {
        flag[l]=1;
        ans+=get(l);
    }
    if(!flag[r+1])
    {
        flag[r+1]=1;
        ans+=get(r+1);
    }
    cout<<ans<<"\n";
}
return 0;
}
```